



**La Universidad Abierta Para Adultos
(UAPA)**

**Equipo #15
Equipo Naranja**

Hugo E. Montero Fulcar
Matricula 100082929

Erasmus José Minaya Taveras
Matricula 100076710

Albert Daniel Montero Ramírez
Matricula 100078731

Luis Ángel Mora Taveras
Matricula 100076432

Fernando Enrique Mesón Acosta
Matricula 100027782

Asignatura:

Estructura de datos y algoritmos
ISW-305

Nombre de la unidad:

Tema

Facilitador:

Zoila Esther Morales Tabares

Índice

1. Introducción
2. Desarrollo de la Problemática - Equipo Naranja
 - 2.1 Análisis del Problema: Almacén Central de Componentes
 - 2.2 Definición de Estructuras de Datos y Clases
 - 2.3 Código Fuente en C++
 - 2.4 Evidencias de Ejecución (Capturas de Pantalla y Video)
3. Ejercicio Común: Control de Inventario Multi-sucursal (Matrices)
 - 3.1 Diseño de la Matriz `stock[6][15]`
 - 3.2 Código Fuente y Métodos de la Clase Almacén
 - 3.3 Evidencias de Ejecución
4. Ejercicio ICPC: Reasignación Óptima
5. Conclusiones
6. Bibliografía y Herramientas Utilizadas

1. Introducción

Este informe detalla la solución técnica a los problemas planteados en la Asignación I, enfocándose en arreglos estáticos y matrices para la gestión de inventarios.

El desarrollo se fundamenta en los principios de la Programación Orientada a Objetos, utilizando el lenguaje C++.

2. Desarrollo de la Problemática - Equipo Naranja

2.1 Análisis del Problema

Se requiere un sistema para una empresa de ensamblaje que gestione componentes electrónicos nacionales e importados. El reto principal consiste en el cálculo dinámico de precios de venta basados en el origen del producto y la detección de niveles críticos de stock.

Fórmula para precios que se aplicó:

Nacionales: $\text{Precio_Costo} + 5\%$

Importados: $\text{Precio_Costo} + 5\% + (27 * \text{Precio_USD})$

2.2 Definición de Estructuras de Datos

Se ha implementado la clase Componente, la cual encapsula los atributos compartidos (código, nombre, cantidad) y atributos específicos según el origen (empresa productora o país de procedencia/precio USD).

2.3 Código Fuente en C++

Enlace al código: [almacen_componentes.cpp](#)

```
#include <iostream>
#include <string>
#include <iomanip>
#include <sstream>
#include <limits>

using namespace std;

const int MAX_COMPONENTES = 50;

class Componente {
private:
    int codigo;
    string nombre;
    double precioCosto;
    int cantidad;
    int nivelMinimo;
    char tipo;
    string paisOrigen;
    double costoImportacion; // Calculado como precioUSD * 27
    string empresaProductora;

public:
    Componente() : codigo(0), precioCosto(0), cantidad(0), nivelMinimo(0), tipo('N') {}

    Componente(int c, string n, double pc, int cant, int minimo, char t, string pais, double
importacion, string empresa) {
        codigo = c; nombre = n; precioCosto = pc; cantidad = cant;
        nivelMinimo = minimo; tipo = t; paisOrigen = pais;
        costoImportacion = importacion; empresaProductora = empresa;
    }

    // Getters y Setters
    int getCodigo() const { return codigo; }
    string getNombre() const { return nombre; }
    int getCantidad() const { return cantidad; }
    int getNivelMinimo() const { return nivelMinimo; }
    char getTipo() const { return tipo; }
```

```

string getPaisOrigen() const { return paisOrigen; }

double calcularPrecioVenta() const {
    double margen = precioCosto * 0.05;
    if (tipo == 'I' || tipo == 'i') return precioCosto + margen + costoImportacion;
    return precioCosto + margen;
}

void mostrar() const {
    cout << "Codigo: " << codigo << " | " << nombre << endl;
    cout << "Stock: " << cantidad << " (Min: " << nivelMinimo << ")" << endl;
    if (tipo == 'I' || tipo == 'i') cout << "Importado de: " << paisOrigen << endl;
    else cout << "Productora: " << empresaProductora << endl;
    cout << "Precio Venta: RD$" << fixed << setprecision(2) << calcularPrecioVenta() << endl;
    cout << "-----" << endl;
}
};

// Implementación de funciones de gestión (registrar, modificar, listar)...

```

```

void cargarDatosIniciales(Componente inventario[], int &total) {
    inventario[total++] = Componente(101, "RTX4060", 19800.00, 10, 5, 'I', "Taiwan", 18910.00, "");
    inventario[total++] = Componente(102, "SSD1TB", 3350.00, 18, 6, 'I', "China", 3538.00, "");
    inventario[total++] = Componente(103, "BoardB650", 7900.00, 12, 4, 'I', "Taiwan", 7808.00, "");
    inventario[total++] = Componente(104, "RAM16GB", 3750.00, 20, 8, 'I', "China", 4026.00, "");
    inventario[total++] = Componente(105, "Ryzen7600", 12800.00, 7, 4, 'I', "USA", 13115.00, "");
    inventario[total++] = Componente(106, "Monitor24", 8450.00, 9, 3, 'N', "", 0.00, "Samsung");
    inventario[total++] = Componente(107, "Fuente650W", 4200.00, 3, 5, 'N', "", 0.00, "Corsair");
    inventario[total++] = Componente(108, "CoolerCPU", 2200.00, 15, 5, 'I', "China", 2318.00, "");
}

```

2.4 Evidencias de Ejecución

Enlace al video y compilación:

Enlace al video de compilación: [Tarea 1 Compilacion Almacen.mp4](#)

```
main.cpp
1
2 #include <iostream>
3 #include <string>
4 #include <iomanip>
5 #include <sstream>
6 #include <limits>
7
8 using namespace std;
9
10 const int MAX_COMPONENTES = 50;
11
12 class Componente {
13 private:
14     int codigo;
15     string nombre;
16     double precioCosto;
17     int cantidad;
18     int nivelMinimo;
19     char tipo;
20     string paisOrigen;
21     double costoImportacion;
22     string empresaProductora;
23
24 public:
25     Componente() {
26         codigo = 0;
```

```
===== SISTEMA DE ALMACEN =====
1. Registrar componente
2. Modificar componente
3. Listar nacionales por precio
4. Listar importados por pais
5. Detectar bajo stock
6. Ver todos
7. Salir
--Tarea 1 ISW 305 Equipo 15--
Seleccione una opcion: 
```

```
===== SISTEMA DE ALMACEN =====
1. Registrar componente
2. Modificar componente
3. Listar nacionales por precio
4. Listar importados por pais
5. Detectar bajo stock
6. Ver todos
7. Salir
--Tarea 1 ISW 305 Equipo 15--
Seleccione una opcion: 3

--- Nacionales por precio ---
Precio minimo en pesos: 10
Codigo: 106
Nombre: Monitor24
Precio costo: RD$8450.00
Cantidad: 9
Nivel minimo: 3
Tipo: Nacional
Empresa productora: Samsung
Precio de venta: RD$8872.50
-----

Codigo: 107
Nombre: Fuente650W
Precio costo: RD$4200.00
Cantidad: 3
Nivel minimo: 5
Tipo: Nacional
Empresa productora: Corsair
Precio de venta: RD$4410.00
-----

Presiona Enter para continuar...
```

--- Componentes con bajo stock ---

Codigo: 107

Nombre: Fuente650W

Precio costo: RD\$4200.00

Cantidad: 3

Nivel minimo: 5

Tipo: Nacional

Empresa productora: Corsair

Precio de venta: RD\$4410.00

Presiona Enter para continuar...

Corrección del código: 1

--- Importados por pais ---

Países disponibles:

- Taiwan

- China

- USA

Pais: USA

Codigo: 105

Nombre: Ryzen7600

Precio costo: RD\$12800.00

Cantidad: 7

Nivel minimo: 4

Tipo: Importado

Pais de origen: USA

Costo de importacion: RD\$13115.00

Precio de venta: RD\$26555.00

Presiona Enter para continuar...

2. Ejercicio #2: Registro de Calificaciones en Bootcamp

2.1 Análisis del Problema

Una academia técnica necesita un sistema que gestione las calificaciones de sus estudiantes en cinco evaluaciones clave. El sistema debe registrar los datos de cada estudiante, calcular automáticamente su promedio, determinar su estado (aprobado/reprobado) con umbral de 70 puntos, filtrar los aprobados y presentar el listado ordenado de mayor a menor promedio.

2.2 Definición de Estructuras de Datos y Clases

Se implementa la clase Estudiante que encapsula todos los datos y comportamientos:

- nombre y matricula: identifican al estudiante.
- calificaciones[5]: arreglo interno que almacena las 5 notas.
- promedio: calculado automáticamente al momento del registro.
- aprobado: booleano determinado según promedio ≥ 70 .
- Método registrar(): punto de entrada que recibe las notas y ejecuta el cálculo.
- Método mostrar(): presenta la fila del estudiante en formato tabular.

El arreglo global estudiantes[MAX_ESTUDIANTES] es el arreglo unidimensional de objetos que centraliza todos los registros, tal como exige el ejercicio.

2.3 Código Fuente en ++


```

#include <iostream>
#include <string>
#include <iomanip>

using namespace std;

const int MAX_ESTUDIANTES = 30;
const int NUM_EVALUACIONES = 5;
const double NOTA_APROBACION = 70.0;

class Estudiante {
private:
    string nombre;
    string matricula;
    double calificaciones[NUM_EVALUACIONES];
    double promedio;
    bool aprobado;

public:
    Estudiante() : promedio(0), aprobado(false) {
        nombre = ""; matricula = "";
        for (int i = 0; i < NUM_EVALUACIONES; i++) calificaciones[i] = 0;
    }

    // a) Registrar notas y calcular promedio automaticamente
    void registrar(string nom, string mat, double notas[]) {
        nombre = nom; matricula = mat;
        double suma = 0;
        for (int i = 0; i < NUM_EVALUACIONES; i++) {
            calificaciones[i] = notas[i];
            suma += notas[i];
        }
        promedio = suma / NUM_EVALUACIONES;           // b) Promedio automatico
        aprobado = (promedio >= NOTA_APROBACION);     // Estado aprobado/reprobado
    }

    // Getters
    string getNombre() const { return nombre; }
    string getMatricula() const { return matricula; }
    double getPromedio() const { return promedio; }
    bool isAprobado() const { return aprobado; }
    double getNota(int i) const { return calificaciones[i]; }

    void mostrar() const {
        cout << left << setw(25) << nombre << setw(14) << matricula;
        for (int i = 0; i < NUM_EVALUACIONES; i++)
            cout << setw(8) << fixed << setprecision(1) << calificaciones[i];
        cout << setw(10) << promedio
            << (aprobado ? "APROBADO" : "REPROBADO") << endl;
    }
};

// Arreglo unidimensional de objetos Estudiante
Estudiante estudiantes[MAX_ESTUDIANTES];
int total = 0;

// a) Registrar estudiantes y notas

```

```

int registrarEstudiantes() {
    int n = 0;
    char continuar = 's';
    cout << "\n==== REGISTRO DE ESTUDIANTES ====" << endl;
    while (n < MAX_ESTUDIANTES && (continuar=='s' || continuar=='S')) {
        string nom, mat;
        double notas[NUM_EVALUACIONES];
        cout << "\nEstudiante #" << (n+1) << endl;
        cout << "Nombre   : "; cin.ignore(); getline(cin, nom);
        cout << "Matricula: "; getline(cin, mat);
        for (int i = 0; i < NUM_EVALUACIONES; i++) {
            do { cout << "Evaluacion " << (i+1) << " (0-100): ";
                cin >> notas[i]; } while (notas[i]<0 || notas[i]>100);
        }
        estudiantes[n].registrar(nom, mat, notas);
        n++;
        if (n < MAX_ESTUDIANTES) {
            cout << "Registrar otro? (s/n): "; cin >> continuar;
        }
    }
    return n;
}

void imprimirCabecera() {
    cout << left << setw(25)<<"Nombre" << setw(14)<<"Matricula";
    for(int i=1;i<=NUM_EVALUACIONES;i++) cout<<setw(8)<<("Eval"+to_string(i));
    cout << setw(10)<<"Promedio" << "Estado" << endl;
    cout << string(95,'-') << endl;
}

// c) Mostrar aprobados con promedio >= 70
void mostrarAprobados() {
    cout << "\n=== ESTUDIANTES APROBADOS (Promedio >= 70) ===" << endl;
    imprimirCabecera();
    bool hay = false;
    for (int i = 0; i < total; i++)
        if (estudiantes[i].isAprobado()) { estudiantes[i].mostrar(); hay=true; }
    if (!hay) cout << "No hay estudiantes aprobados." << endl;
}

// d) Ordenar por promedio (burbuja, mayor a menor)
void ordenarPorPromedio() {
    for (int i = 0; i < total-1; i++)
        for (int j = 0; j < total-1-i; j++)
            if (estudiantes[j].getPromedio() < estudiantes[j+1].getPromedio()) {
                Estudiante temp = estudiantes[j];
                estudiantes[j] = estudiantes[j+1];
                estudiantes[j+1] = temp;
            }
}

void mostrarTodos() {
    cout << "\n=== LISTADO GENERAL (Por Promedio) ===" << endl;
    imprimirCabecera();
    for (int i = 0; i < total; i++) estudiantes[i].mostrar();
}

int main() {
    int opcion;

```

```

cout << "===== " << endl;
cout << "  SISTEMA DE CALIFICACIONES BOOTCAMP" << endl;
cout << "===== " << endl;
do {
cout << "\n1. Registrar estudiantes y notas" << endl;
cout << "2. Mostrar aprobados" << endl;
cout << "3. Ordenar y mostrar todos por promedio" << endl;
cout << "4. Salir" << endl;
cout << "Opcion: "; cin >> opcion;
switch(opcion) {
    case 1: total = registrarEstudiantes(); break;
    case 2: if(total==0) cout<<"Registre estudiantes primero."<<endl;
            else mostrarAprobados(); break;
    case 3: if(total==0) cout<<"Registre estudiantes primero."<<endl;
            else { ordenarPorPromedio(); mostrarTodos(); } break;
    case 4: cout<<"Saliendo..."<<endl; break;
    default: cout<<"Opcion invalida."<<endl;
}
} while(opcion != 4);
return 0;
}

```

2.4 Explicación de Funciones Principales

Funcion / Metodo	Descripcion
registrar()	Recibe nombre, matrícula y arreglo de 5 notas. Calcula promedio y determina estado.
registrarEstudiantes()	Ciclo interactivo que captura datos hasta MAX_ESTUDIANTES o que el usuario decida parar.
mostrarAprobados()	Recorre el arreglo y muestra sólo los estudiantes con promedio ≥ 70 .
ordenarPorPromedio()	Algoritmo de burbuja que reordena el arreglo de mayor a menor promedio.
mostrarTodos()	Imprime todos los estudiantes en formato tabular con cabecera.

2.5 Flujo del Sistema

El menú principal ofrece cuatro opciones independientes:

- Opcion 1 – Registrar: captura iterativa de nombre, matrícula y 5 evaluaciones por estudiante con validación de rango (0-100).
- Opcion 2 – Aprobados: lista filtrada de estudiantes con promedio mayor o igual a 70, indicando estado APROBADO.
- Opcion 3 – Ordenado: aplica el ordenamiento burbuja y muestra el listado completo de mayor a menor promedio.

- Opcion 4 – Salir: cierra el sistema.

2.X.6 Evidencias de Ejecución

```
=====
      SISTEMA DE CALIFICACIONES - BOOTCAMP
=====

--- MENU PRINCIPAL ---
1. Registrar estudiantes y notas
2. Mostrar estudiantes aprobados
3. Ordenar y mostrar todos por promedio
4. Salir
```

Captura 1: Menú principal del sistema con las cuatro opciones disponibles.

```
Estudiante #1
Nombre completo: Erasmo Jose Minaya Taveras
Matricula      : 100076710
Evaluacion 1 (0-100): 100
Evaluacion 2 (0-100): 100
Evaluacion 3 (0-100): 90
Evaluacion 4 (0-100): 98
Evaluacion 5 (0-100): 97
Registrar otro estudiante? (s/n): s

Estudiante #2
Nombre completo: Lamine Yamal
Matricula      : 100078910
Evaluacion 1 (0-100): 80
Evaluacion 2 (0-100): 80
Evaluacion 3 (0-100): 90
Evaluacion 4 (0-100): 100
Evaluacion 5 (0-100): 50
Registrar otro estudiante? (s/n): s

Estudiante #3
Nombre completo: Pedro Pascal
Matricula      : 100012345
Evaluacion 1 (0-100): 70
Evaluacion 2 (0-100): 41
Evaluacion 3 (0-100): 92
Evaluacion 4 (0-100): 50
Evaluacion 5 (0-100): 01
Registrar otro estudiante? (s/n):
s
```

Captura 2: Registro de tres estudiantes con sus cinco calificaciones y cálculo automático de promedio.

```

===== ESTUDIANTES APROBADOS (Promedio >= 70) =====
Nombre | Matricula | Eval1 Eval2 Eval3 Eval4 Eval5 | Promedio | Estado
-----|-----|-----|-----|-----|-----|-----|-----|-----|-----
Erasmus Jose Minaya Taveras | 100076710 | 100.0 100.0 90.0 98.0 97.0 | 97.0 | APROBADO
Urmama Cockaba | 100069674 | 100.0 100.0 50.0 69.0 | 83.8 | APROBADO
Lamine Yamal | 100078910 | 80.0 80.0 90.0 100.0 50.0 | 80.0 | APROBADO

```

Captura 3: Listado de estudiantes aprobados filtrado por promedio >= 70.

```

===== LISTADO GENERAL (Ordenado por Promedio) =====
Nombre | Matricula | Eval1 Eval2 Eval3 Eval4 Eval5 | Promedio | Estado
-----|-----|-----|-----|-----|-----|-----|-----|-----|-----
Erasmus Jose Minaya Taveras | 100076710 | 100.0 100.0 90.0 98.0 97.0 | 97.0 | APROBADO
Urmama Cockaba | 100069674 | 100.0 100.0 50.0 69.0 | 83.8 | APROBADO
Lamine Yamal | 100078910 | 80.0 80.0 90.0 100.0 50.0 | 80.0 | APROBADO
Ernesto De La Cruz | 100004129 | 70.0 8.0 80.0 9.0 90.0 | 51.4 | REPROBADO
Pedro Pascal | 100012345 | 70.0 41.0 92.0 50.0 1.0 | 50.8 | REPROBADO

```

Captura 4: Listado completo ordenado de mayor a menor promedio usando algoritmo de burbuja.

Enlace al video de compilación: 📺 2026-05-10 20-49-01.mp4

3. Ejercicio Común: Control de Inventario Multi-sucursal

Se gestiona una matriz de 6 sucursales por 15 tipos de productos, permitiendo una visión global de la existencia de la empresa.

3.1 Análisis de Funcionalidades

Matriz stock[6][15]: Almacenamiento estático de existencias.

Detección de Agotados: Recorrido exhaustivo de la matriz buscando valores iguales a cero.

Alerta por Umbral: El sistema permite al usuario definir un límite (umbral) y filtra todos los productos que requieren reposición.

3.2 Código Fuente (C++)

Enlace al código: [inventario_multisucursal.cpp](#)

```
#include <iostream>
#include <iomanip>
#include <string>
#include <sstream>

using namespace std;

const int TOTAL_ALMACENES = 6;
const int TOTAL_PRODUCTOS = 15;

class Almacen {
private:
    int stock[TOTAL_ALMACENES][TOTAL_PRODUCTOS];

public:
    Almacen() {
        // Inicialización de matriz con datos de prueba
        int inicial[TOTAL_ALMACENES][TOTAL_PRODUCTOS] = {
            {40, 25, 18, 35, 42, 22, 30, 28, 14, 26, 50, 60, 8, 6, 20},
            {32, 20, 12, 30, 38, 18, 24, 21, 11, 19, 45, 48, 6, 5, 16},
            // ... resto de datos iniciales
        };
    };
};
```

```

    for(int i=0; i<TOTAL_ALMACENES; i++)
        for(int j=0; j<TOTAL_PRODUCTOS; j++) stock[i][j] = inicial[i][j];
}

void emitirAlertasPorUmbral(int umbral) const {
    cout << "\n===== ALERTAS POR UMBRAL (<= " << umbral << ") =====" << endl;
    for (int i = 0; i < TOTAL_ALMACENES; i++) {
        for (int j = 0; j < TOTAL_PRODUCTOS; j++) {
            if (stock[i][j] <= umbral) {
                cout << "Almacen " << (i + 1) << " - P" << (j + 1) << ": " << stock[i][j] << " unidades." << endl;
            }
        }
    }
}

// Métodos adicionales: registrarExistencia, obtenerAlmacenConMenorStock...
};

```

4. EVIDENCIAS DE COMPILACIÓN

```
===== INVENTARIO MULTISUCURSAL =====
1. Registrar existencia
2. Ver matriz de stock
3. Detectar productos agotados
4. Almacen con menor stock
5. Emitir alertas por umbral
6. Salir
Seleccione una opcion: 2
```

Captura 1: Menú principal y registro de componentes.

```
===== MATRIZ DE STOCK (Almacen x Producto) =====
      P 1 P 2 P 3 P 4 P 5 P 6 P 7 P 8 P 9 P10 P11 P12 P13 P14 P15
A1 -> 40 25 18 35 42 22 30 28 14 26 50 60 8 6 20
A2 -> 32 20 12 30 38 18 24 21 11 19 45 48 6 5 16
A3 -> 28 16 10 22 30 15 20 18 9 15 37 40 5 4 14
A4 -> 36 22 15 29 35 19 26 23 13 21 42 52 7 6 18
A5 -> 24 14 8 18 26 12 16 15 7 12 30 35 4 3 11
A6 -> 30 19 13 25 33 17 22 20 10 18 39 44 6 5 15

Leyenda de productos:
P1: Aceite de motor
P2: Llantas
P3: Baterias
P4: Filtros de aire
P5: Bujias
P6: Liquido de frenos
P7: Refrigerante
P8: Pastillas de freno
P9: Amortiguadores
P10: Correas
P11: Limpiaparabrisas
P12: Bombillos
P13: Alternadores
P14: Radiadores
P15: Sensores
```

Captura 2: Matriz de stock multisucursal visualizada en consola.


```
===== INVENTARIO MULTISUCURSAL =====
1. Registrar existencia
2. Ver matriz de stock
3. Detectar productos agotados
4. Almacen con menor stock
5. Emitir alertas por umbral
6. Salir
Seleccione una opcion: 4
Numero de producto (1-15): 1
El almacen con menor stock de Aceite de motor es el almacen 5 con 24 unidades.
```

```
6. Salir
Seleccione una opcion: 5
Umbral de alerta: 10

===== ALERTAS POR UMBRAL (<= 10) =====
Almacen 1 - Alternadores (P13): 8
Almacen 1 - Radiadores (P14): 6
Almacen 2 - Alternadores (P13): 6
Almacen 2 - Radiadores (P14): 5
Almacen 3 - Baterias (P3): 10
Almacen 3 - Amortiguadores (P9): 9
Almacen 3 - Alternadores (P13): 5
Almacen 3 - Radiadores (P14): 4
Almacen 4 - Alternadores (P13): 7
Almacen 4 - Radiadores (P14): 6
Almacen 5 - Baterias (P3): 8
Almacen 5 - Amortiguadores (P9): 7
Almacen 5 - Alternadores (P13): 4
Almacen 5 - Radiadores (P14): 3
Almacen 6 - Amortiguadores (P9): 10
Almacen 6 - Alternadores (P13): 6
Almacen 6 - Radiadores (P14): 5
```

Captura 3: Alerta de bajo stock activada por umbral configurable.

Enlace al video y compilación:

Enlace al video de compilación:  Punto_3.mp4

4. EJERCICIO DE MATRICES PARA ICPC: REASIGNACIÓN ÓPTIMA

Este ejercicio realiza una optimización logística mediante la redistribución de productos entre almacenes para cubrir faltantes críticos, minimizando el costo total basado en una matriz de distancias.

4.1 Lógica del Algoritmo

Se utiliza un algoritmo de optimización que busca el origen con excedente más cercano a cada destino en déficit. El costo se calcula como la cantidad por la distancia.

Código C++ [reasnacion_optima.cpp](#)

Código Python [reasnacion_optima.py](#)

4.2 Código Fuente en C++

```
#include <iostream>
#include <string>
#include <iomanip>

using namespace std;

const int ALMACENES = 6;
const int PRODUCTOS = 15;

struct Movimiento {
    int producto;
    int almacenOrigen;
    int almacenDestino;
    int cantidadMovida;
    double costoAsociado;
};

int realizarReasnacion(int stock[ALMACENES][PRODUCTOS], int
distancia[ALMACENES][ALMACENES], int umbral[PRODUCTOS], Movimiento movimientos[],
double &costoTotal) {
    int totalMovimientos = 0;
    for (int p = 0; p < PRODUCTOS; p++) {
```

```

    for (int dest = 0; dest < ALMACENES; dest++) {
        if (stock[dest][p] < umbral[p]) {
            int necesita = umbral[p] - stock[dest][p];
            // Lógica de búsqueda de origen más cercano...
        }
    }
}
return totalMovimientos;
}

```

4.3 Código Fuente en Python (Requisito ICPC)

```

def realizar_reasignacion():
    # Implementación de matrices de stock y distancia
    stock = [[25, 16, 8, ...], ...]
    distancia = [[0, 12, 8, ...], ...]
    umbral = [10, 10, 9, ...]

    # Lógica de optimización logística
    print("Resumen de movimientos y costos calculados exitosamente.")

```

5. EVIDENCIAS DE COMPILACIÓN

Enlace al video y compilación:

Enlace al video de compilación:  Punto 4.mp4

```
===== ESTADO INICIAL =====
Stock (filas=almacenes, columnas=productos):
      P01 P02 P03 P04 P05 P06 P07 P08 P09 P10 P11 P12 P13 P14 P15
A1 -> 25 16  8 14 11 13 10 15  9 12 18 22  6  5  8
A2 ->  8  6 15 10  7  5  9  6 14  8  7 10 12  9  6
A3 -> 18 20  4 16 15 14 17  5  7 18  9  8  4  6  5
A4 ->  6  7 10  5  8  6  5 12 11  4  6  7 10  8  7
A5 -> 22 14 12 18 16 17 13 19 10 15 12 11  8 10  9
A6 ->  9 11  7  6  5  8  6  9 13  7 10  9  5  4  6

Distancias entre almacenes:
      A1  A2  A3  A4  A5  A6
A1 ->  0 12  8 15 20 10
A2 -> 12  0  9  7 18 14
A3 ->  8  9  0 11 16  6
A4 -> 15  7 11  0 10 13
A5 -> 20 18 16 10  0 12
A6 -> 10 14  6 13 12  0
```

Distancias entre almacenes:

	A1	A2	A3	A4	A5	A6
A1 ->	0	12	8	15	20	10
A2 ->	12	0	9	7	18	14
A3 ->	8	9	0	11	16	6
A4 ->	15	7	11	0	10	13
A5 ->	20	18	16	10	0	12
A6 ->	10	14	6	13	12	0

===== ESTADO FINAL =====

Stock (filas=almacenes, columnas=productos):

	P01	P02	P03	P04	P05	P06	P07	P08	P09	P10	P11	P12	P13	P14	P15
A1 ->	25	16	9	14	11	13	10	11	9	12	12	17	7	7	7
A2 ->	10	10	9	9	8	8	8	9	14	9	10	10	8	7	7
A3 ->	15	16	9	13	11	11	15	9	8	15	10	10	7	7	7
A4 ->	10	10	10	9	8	8	8	9	11	9	10	10	9	7	7
A5 ->	18	11	10	15	16	15	11	19	10	10	10	10	7	7	7
A6 ->	10	11	9	9	8	8	8	9	12	9	10	10	7	7	6

===== ESTADO FINAL =====

Stock (filas=almacenes, columnas=productos):

	P01	P02	P03	P04	P05	P06	P07	P08	P09	P10	P11	P12	P13	P14	P15
A1 ->	25	16	9	14	11	13	10	11	9	12	12	17	7	7	7
A2 ->	10	10	9	9	8	8	8	9	14	9	10	10	8	7	7
A3 ->	15	16	9	13	11	11	15	9	8	15	10	10	7	7	7
A4 ->	10	10	10	9	8	8	8	9	11	9	10	10	9	7	7
A5 ->	18	11	10	15	16	15	11	19	10	10	10	10	7	7	7
A6 ->	10	11	9	9	8	8	8	9	12	9	10	10	7	7	6

===== RESUMEN DE MOVIMIENTOS =====

#	Producto	Origen	Destino	Cantidad	Costo
1	Aceite de motor	A3	A2	2	18.00
2	Aceite de motor	A5	A4	4	40.00
3	Aceite de motor	A3	A6	1	6.00
4	Llantas	A3	A2	4	36.00
5	Llantas	A5	A4	3	30.00
6	Baterias	A2	A1	1	12.00
7	Baterias	A2	A3	5	45.00
8	Baterias	A5	A6	2	24.00
9	Filtros de aire	A2	A4	1	7.00
10	Filtros de aire	A5	A4	3	30.00
11	Filtros de aire	A3	A6	3	18.00
12	Bujias	A3	A2	1	9.00
13	Bujias	A3	A6	3	18.00
14	Liquido de frenos	A3	A2	3	27.00
15	Liquido de frenos	A5	A4	2	20.00
16	Refrigerante	A2	A4	1	7.00
17	Refrigerante	A5	A4	2	20.00
18	Refrigerante	A3	A6	2	12.00
19	Pastillas de freno	A4	A2	3	21.00
20	Pastillas de freno	A1	A3	4	32.00
21	Amortiguadores	A6	A3	1	6.00
22	Correas	A3	A2	1	9.00
23	Correas	A5	A4	5	50.00
24	Correas	A3	A6	2	12.00
25	Limpiaparabrisas	A1	A2	3	36.00
26	Limpiaparabrisas	A1	A3	1	8.00
27	Limpiaparabrisas	A5	A4	2	20.00
28	Limpiaparabrisas	A1	A4	2	30.00
29	Bombillos	A1	A3	2	16.00
30	Bombillos	A5	A4	1	10.00
31	Bombillos	A1	A4	2	30.00
32	Bombillos	A1	A6	1	10.00
33	Alternadores	A2	A1	1	12.00
34	Alternadores	A2	A3	3	27.00
35	Alternadores	A5	A6	1	12.00
36	Alternadores	A4	A6	1	13.00
37	Radiadores	A2	A1	2	24.00
38	Radiadores	A4	A3	1	11.00
39	Radiadores	A5	A6	3	36.00
40	Sensores	A1	A2	1	12.00
41	Sensores	A5	A3	2	32.00

Costo total de reasignacion: 848.00

4. Conclusiones

La implementación de arreglos y matrices estáticas permite un control riguroso de la información en sistemas de pequeña y mediana escala. El uso de POO facilita la mantenibilidad del código y la escalabilidad de las reglas de negocio, como los cálculos de impuestos y divisas.

5. Bibliografía y Herramientas

OnlineGBD